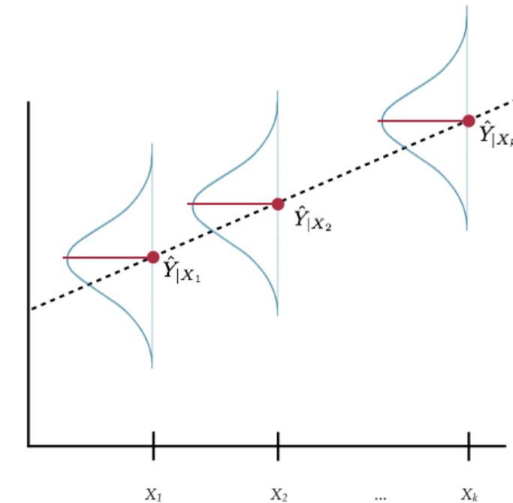
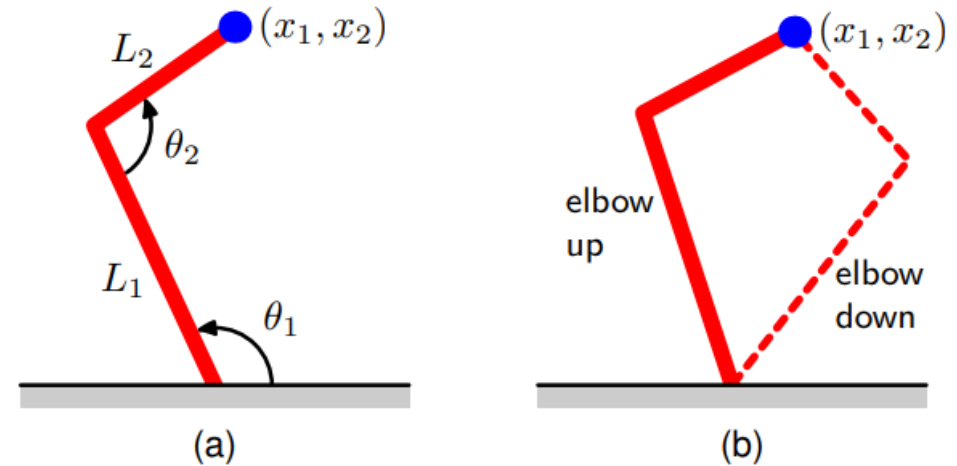


05 | Mixture Density Networks

Why Gaussian is Not Enough

- 기존 회귀의 가정
 - ✓ $p(t|\mathbf{x}, \mathbf{w}) = \mathcal{N}(t|y(\mathbf{x}, \mathbf{w}), \sigma^2)$
 - ✓ 하나의 평균, 하나의 분산, unimodal
- Inverse Problem
 - ✓ Forward problem: $(\theta_1, \theta_2) \rightarrow (x_1, x_2)$ 해가 유일
 - ✓ Inverse problem: $(x_1, x_2) \rightarrow (\theta_1, \theta_2)$ 해가 여러 개
 - ✓ 즉, 입력 하나에 대해 출력이 여러 개 가능
- Gaussian 가정의 한계
 - ✓ MSE 기반 회귀는 조건부 평균만 학습
 - ✓ 여러 해가 있으면 평균을 예측하며, 실제 해가 아님



05 | Mixture Density Networks

Example

- 자율주행 차량의 미래 경로 예측
 - ✓ 자율주행차가 앞 차량의 미래 위치를 예측해야 함
 - ✓ 입력: 현재 위치, 속도, 도로 구조
 - ✓ 출력: 3초 뒤 차량 위치
- But
 - ✓ 앞 차량은 좌회전, 우회전 할 수 있음
 - ✓ MSE는 **평균** 위치를 예측 하지만 차가 도로 중앙으로 뚫고 가는 꼴이 될 수 있음
- MDN 사용하면?
 - ✓ $\pi_1 = 0.4 \rightarrow$ 좌회전, $\pi_2 = 0.6$ 우회전 그리고 각 μ_k 는 각각 실제 경로 위치
 - ✓ 즉. 여러 미래 가능성을 동시에 모델링 가능

05 | Mixture Density Networks

How to Implement?

- General Framework

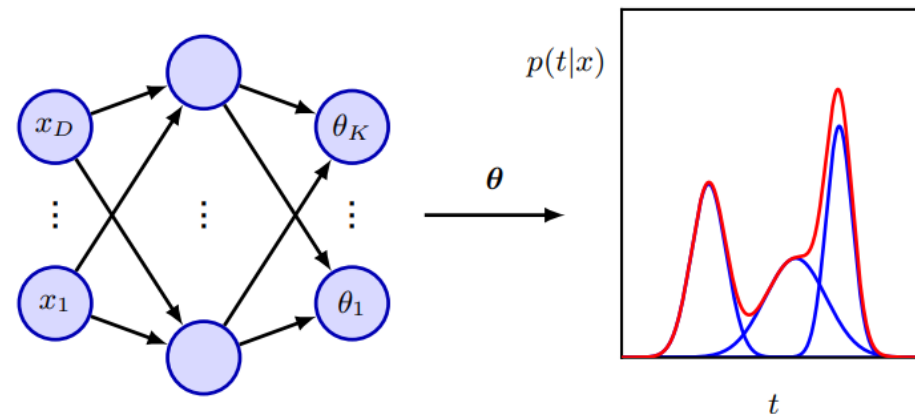
- ✓ 단일 Gaussian 대신 Mixture Model 사용
- ✓ 조건부 확률을 Gaussian mixture로 모델링
- ✓ Multimodal 조건부 분포 모델링 가능

- Conditional Distribution

- ✓ 신경망이 (π_k, μ_k, σ_k) 를 직접 출력
- ✓
$$p(\mathbf{t}|\mathbf{x}) = \sum_{k=1}^K \pi_k(\mathbf{x}) \mathcal{N}(\mathbf{t}|\boldsymbol{\mu}_k(\mathbf{x}), \sigma_k^2(\mathbf{x}))$$

- Output Constraints & Activations

- ✓ $\pi_k \rightarrow \text{Softmax}$ (합 = 1)
- ✓ $\sigma_k \rightarrow \exp()$ (양수 보장)
- ✓ $\mu_k \rightarrow \text{Identity}$



- Training Objective

- ✓
$$E(\mathbf{w}) = - \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k(\mathbf{x}_n, \mathbf{w}) \mathcal{N}(\mathbf{t}_n | \boldsymbol{\mu}_k(\mathbf{x}_n, \mathbf{w}), \sigma_k^2(\mathbf{x}_n, \mathbf{w})) \right\}$$

- ✓ Negative Log-Likelihood 최소화
- ✓ 조건부 확률 분포 전체 학습

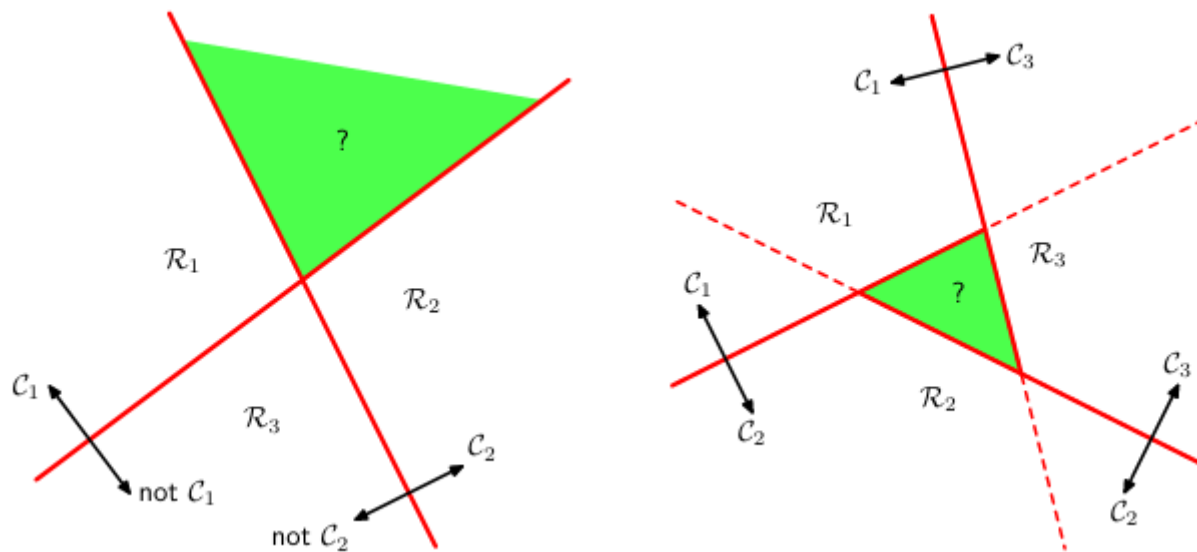
- Why Not Widely Used?

- ✓ 구현 복잡 (log-sum-exp 안정화 필요)
- ✓ 최근에는 Flow / Diffusion / VAE 등 대안 모델 등장

02 | Discriminant Functions

Multiple Classes: Ambiguity Problems

- Issues with Binary Classifier Combinations
 - ✓ One-versus-the-rest (일대다 방식)
 - 문제점: 두 클래스에 동시에 속하거나, 어디에도 속하지 않는 모호한 영역 발생 (그림 좌측 녹색 영역)
 - $K - 1$ 개의 분류기 사용: C_k vs not C_k
 - ✓ One-versus-one (일대일 방식)
 - $K(K - 1)/2$ 개의 분류기 사용 (모든 쌍 비교)
 - 문제점: 다수결 투표 시 모순 발생 가능 (그림 우측 녹색 영역)



02 | Discriminant Functions

K-Class Discriminant

- Solution: Single K-class Discriminant

- ✓ K 개의 선형 함수:

$$y_k(x) = w_k^T x + w_{k0}$$

- ✓ 결정 함수:

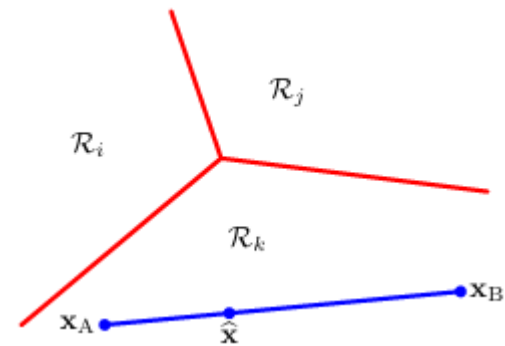
$$\text{class}(x) = \text{argmax}_k y_k(x)$$

- Decision Rule

- ✓ Decision boundary: $y_k(x) = y_j(x)$ 인 초평면

$$(w_k - w_j)x + (w_{k0} - w_{j0}) = 0$$

- ✓ Convexity (볼록성): 결정 영역 R_k 는 단일 연결된 볼록(singly connected and convex) 형태
 - 영역 내 임의의 두 점을 연결한 선분은 영역을 벗어나지 않음





Deep Learning Foundations and Concepts 5주차

7. Gradient Descent

김호중 / 2026.03.05



Computational Data Science LAB



CONTENTS

1. Error Surfaces
 2. Gradient Descent Optimization
 3. Advanced Convergence Techniques
 4. Normalization Techniques
- 
- 

01 | Error Surfaces

Local Quadratic Approximation

- 고차원 에러 표면의 해석 한계

- ✓ 심층 신경망은 수백만~수십억 개의 파라미터를 가지며, 비볼록(Non-convex) 에러 함수 $E(w)$ 의 전체 지형을 수학적으로 파악하는 것은 불가능
- ✓ 따라서 현재 파라미터 위치 $\hat{w}$ 주변의 국소적 지형만을 테일러 전개를 통해 2차 함수(포물선/타원체)로 근사화하여 분석

❖ 다변수 테일러 전개(Multivariable Taylor Expansion)는 한 점에서의 함수값과 도함수(미분값)들을 이용해, 그 점 근처의 함수값을 근사하는 방법 $f(x) = f(a) + f'(a)(x - a) + \frac{1}{2}f''(a)(x - a)^2 + \dots$

- 국소적 2차 근사 수식

$$E(w) \simeq E(\hat{w}) + (w - \hat{w})^T b + \frac{1}{2}(w - \hat{w})^T H(w - \hat{w})$$

- ✓ $E(\hat{w})$: 현재 위치에서의 실제 에러값 (Base Error 스칼라)
- ✓ $(w - \hat{w})$: 현재 위치에서 새로운 위치로 내딛는 이동 벡터 (Step Vector)
- ✓ $b = \nabla E(\hat{w})$: 1차 미분 (Gradient 벡터). 현재 위치에서 가장 가파른 오르막의 방향과 그 기울기를 의미
- ✓ $H = \nabla \nabla E(w)$: 2차 미분 (Hessian 대칭 행렬). 공간상 지형이 어떻게 휘어져 있는지를 나타내는 차원 간 곡률

01 | Error Surfaces

Eigenvalue Analysis & Saddle Points

- 정류점(Stationary Point)에서의 에러 변화량

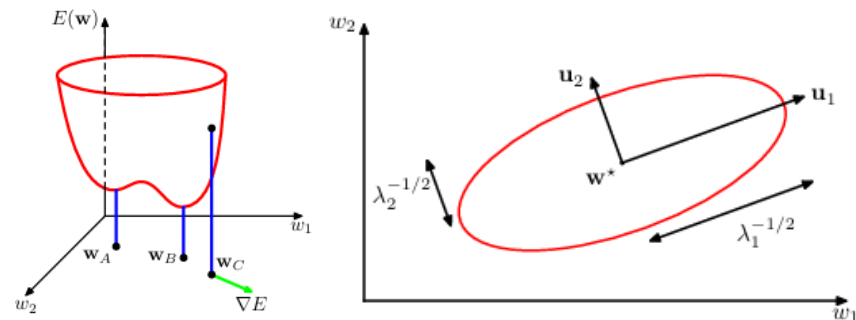
- ✓ 기울기가 0인 지점($b = 0$)에 도달 시, Hessian 행렬을 고유값 분해($Hu_i = \lambda_i u_i$)하여 대입하면 수식이 단순한 스칼라 합으로 정리됨

$$\Delta E \approx \frac{1}{2} \sum \lambda_i \alpha_i^2$$

- ✓ α_i^2 : i 번째 고유벡터 주축 방향으로 이동한 거리의 제곱 (항상 양수)
- ✓ λ_i : i 번째 축 방향의 고유값 (해당 방향이 얼마나 가파르게 휘었는지 나타내는 곡률)

- Local Minima vs Saddle Point

- ✓ Local Minima (지역 최소값) : 모든 $\lambda_i > 0$. 어떤 방향(α_i)으로 가든 오차가 무조건 증가(+)하므로 완벽한 바닥 형태 (예: $\lambda_1 = 2, \lambda_2 = 4$)
- ✓ Saddle Point (안장점) : λ_i 부호가 혼재된 상태. 양수 방향으로 오르막이나, 음수 방향으로 내리막이 존재 (예: $\lambda_1 = 2, \lambda_2 = -4$)
- ✓ 수백만 차원 공간에서는 모든 고유값이 우연히 양수일 확률이 0에 수렴함
 - 따라서 최적화 도중 마주치는 학습 지연의 핵심 원인은 지역 최소값이 아닌 '안장점'



02 | Gradient Descent Optimization

Efficiency of Gradient Information

- 2차 최적화(Newton's Method)의 연산량 한계

$$w_{\text{new}} = w_{\text{old}} - H^{-1} \nabla E$$

- ✓ 2차 근사식을 이용해 한 번에 최적점으로 점프(2차 근사 수식을 행렬/벡터 미분 공식 사용해 w 에 대해 미분)하려면 크기가 W 인 헤시안 행렬의 역행렬(H^{-1}) 계산이 반드시 필요
- ✓ 선형대수학적으로 역행렬 연산량은 $O(W^3)$ (가우스 소거법 등으로 역행렬 계산시 $N \times N \times N$ 연산량)
 - 파라미터가 1,750억 개인 거대 언어 모델(LLM) 등에 적용할 경우 우주적 스케일의 시간이 소요되어 계산 불가능
- 역전파(Backpropagation)와 1차 미분의 효율성
 - ✓ 단순 1차 미분 기반의 연쇄 법칙(Chain rule)을 활용하여, 순방향/역방향 스윙 단 1번 만으로 모든 가중치의 미분값을 동시에 추출 (1스텝당 연산량 $O(W)$)
 - ✓ 최적점 도달까지 필요한 반복 횟수(Step)가 파라미터 수 W 에 비례한다고 보통 수학적으로 가정할 때, 총 연산량은 $O(W^2)$ 로 획기적으로 단축되어 GPU 기반의 현실적 학습이 가능해짐

02 | Gradient Descent Optimization

Batch vs Stochastic Gradient Descent

- Batch Gradient Descent

$$w(\tau) = w^{(\tau-1)} - \eta \nabla E(w^{(\tau-1)})$$

- ✓ η (Learning Rate): 내리막 방향으로 얼마나 이동할지 결정하는 보폭 크기
- ✓ 전체 데이터셋 N 개(예: 120만 장)의 에러를 모두 평균 내어 완벽하게 정확한 그래디언트 ∇E 를 계산
- ✓ 그러나 1회 업데이트(1 Step)에 수십 분 이상이 소요되어 대규모 모델 학습에 원천적으로 부적합

- Stochastic Gradient Descent

- ✓ 전체가 아닌 무작위 단일 데이터 x_n 1개만을 이용하여 그래디언트를 근사 $\rightarrow$ 연산 속도 N 배 향상
- ✓ Gradient Noise 효과: 모집단의 진짜 기울기를 표본 1개로 추정하므로 ∇E 벡터에 필연적으로 분산(노이즈)이 발생하여 궤적이 지그재그로 흔들림
- ✓ 이 무작위한 흔들림이 오히려 모델을 Saddle point나 얇은 웅덩이에서 강제로 튕겨 나가게 만드는 원동력으로 작용

Algorithm 7.1: Stochastic gradient descent

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$
Error function per data point $E_n(\mathbf{w})$
Learning rate parameter η
Initial weight vector $\mathbf{w}$
Output: Final weight vector $\mathbf{w}$

```
 $n \leftarrow 1$ 
repeat
   $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_n(\mathbf{w})$  // update weight vector
   $n \leftarrow n + 1 \pmod{N}$  // iterate over data
until convergence
return  $\mathbf{w}$ 
```

02 | Gradient Descent Optimization

Mini-batch & The Law of Diminishing Returns

- Mini-batch SGD
 - ✓ BGD의 치명적인 연산 비용 문제와 SGD의 극단적인 방향성 노이즈 문제를 절충하는 방법
 - ✓ 하드웨어(GPU)의 행렬 연산 병렬 처리 효율을 극대화하기 위해 보통 32~256개 단위의 데이터 묶음(Mini-batch)으로 그래디언트 추정
- 수확 체감의 법칙 (The Law of Diminishing Returns)
 - ✓ 통계학적으로 표본 크기 N 이 증가할 때 추정 오차(표준오차)의 감소율은 $1/\sqrt{N}$ 에 비례
 - ✓ 배치 크기를 100에서 10,000으로 100배 늘려 엄청난 연산량을 폭발시켜도, 실제 기울기 정확도 오차는 단 1/10밖에 줄어들지 않음
 - ✓ 맹목적으로 큰 배치를 사용하는 것은 극심한 연산(비용) 낭비이며 모델 일반화 성능에도 악영향을 미침

Algorithm 7.2: Mini-batch stochastic gradient descent

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$
Batch size B
Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
Learning rate parameter η
Initial weight vector $\mathbf{w}$
Output: Final weight vector $\mathbf{w}$

```
 $n \leftarrow 1$ 
repeat
   $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{n:n+B-1}(\mathbf{w})$  // weight vector update
   $n \leftarrow n + B$ 
  if  $n > N$  then
    shuffle data
     $n \leftarrow 1$ 
  end if
until convergence
return  $\mathbf{w}$ 
```

02 | Gradient Descent Optimization

Parameter Initialization & Symmetry Breaking

- 대칭성 파괴(Symmetry Breaking) 실패
 - ✓ 가중치(w)를 편의상 모두 0으로 초기화하면, 동일한 입력을 받는 동일 층의 은닉 노드들이 완벽히 동일한 출력값과 동일한 역전파 기울기를 가짐
 - ✓ 은닉 노드가 수천 개라도 마치 단 1개의 노드처럼 동일하게 업데이트되므로 학습이 붕괴됨→무작위(Random) 초기화가 절대적으로 필요
- He Initialization : 헤이밍 초기화 (for *ReLU*)
 - ✓ 망이 깊어질 때 노드를 통과하며 데이터 분포의 분산(Scale)이 폭발하거나 0으로 소멸하는 것을 방지하는 정교한 초기화 수식
 - ✓ 분산 유지 조건 공식(출력)
$$var[z] = \frac{M}{2} \epsilon^2 \lambda^2$$
 - ✓ (M : 이전 층의 노드 수, λ^2 : 입력 데이터의 원본 분산, ϵ : 가중치의 분산)
 - ✓ *ReLU* 활성화 함수는 음수 값을 모두 버려 분산을 반토막 내므로, 이를 상쇄하기 위해 1/2항이 수식에 필연적으로 등장함
 - ✓ 만약 입/출력 분산을 일정하게 1로 보존하려면, 역으로 가중치를 가우시안 표준편차 $\epsilon = \sqrt{\frac{2}{M}}$ 을 사용하여 무작위 추출

03 | Advanced Convergence Techniques

Ill-conditioning Problem

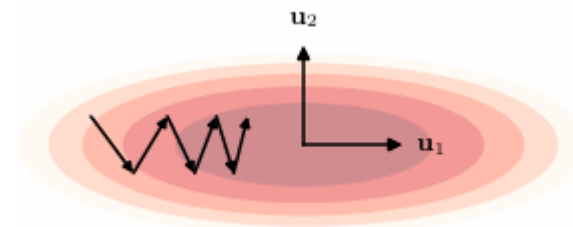
- Definition of Condition Number (조건수)

- ✓ 에러 표면의 가장 가파른 벽의 기울기(λ_{max})과 가장 완만한 바닥의 기울기(λ_{min})의 비율
- ✓ 현실의 데이터는 피쳐 간 스케일이나 상관관계 차이로 인해, 에러 표면이 거의 항상 조건수가 매우 높은 좁고 긴 계곡 형태를 띠

- 진동(Oscillation) 현상

$$E = 50w_1^2 + 0.5w_2^2$$

- ✓ (조건수 $100/1 = 100$). 학습률 $\eta = 0.015$ 가정
- ✓ 그래디언트 스텝 계산 시, 가파른 w_1 축 방향으로 $+0.25 \rightarrow -0.5 \rightarrow +0.25$ 처럼 탁구공처럼 좌우 벽을 번갈아 치는 극심한 진동 발생
- ✓ 에너지를 좌우 진동에 모두 낭비하여, 정작 목표 방향인 완만한 w_2 축으로는 거의 전진하지 못해 수렴이 극도로 지연



03 | Advanced Convergence Techniques

Momentum Algorithm

• 모멘텀 알고리즘

$$\Delta w^{(\tau-1)} = -\eta \nabla E(w^{(\tau-1)}) + \mu \Delta w^{(\tau-2)}$$

- ✓ 현재 내딛는 벡터 = (현재 기울기 × 학습률) + (과거 이동 벡터 × 관성 계수 μ)
- ✓ 과거 궤적(Velocity)을 기억하여 현재 스텝에 반영. 통상 $\mu = 0.9$ 사용

• Mechanisms of *Momentum*

- ✓ 가파른 축 (진폭 상쇄, Damping)
 - 좁은 계곡 벽을 치며 기울기 부호 (+, -)가 매번 교차
 - 직전 과거의 이동 방향 (+)과 현재 기울기 부호 (-)가 반대이므로 서로 덧셈되며 값이 깎여 진동폭이 급감
- ✓ 완만한 축 (가속, Acceleration)
 - 계곡 바닥을 향한 기울기 부호는 일관되게 일정
 - 한 방향이 계속 누적되어 무한 등비급수에 따라 유효 학습률이 $\frac{\eta}{1-\mu}$ 배

Algorithm 7.3: Stochastic gradient descent with momentum

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$
 Batch size B
 Error function per mini-batch $E_{n:n+B-1}(w)$
 Learning rate parameter η
 Momentum parameter μ
 Initial weight vector w

Output: Final weight vector w

```

n ← 1
Δw ← 0
repeat
  Δw ← -η∇En:n+B-1(w) + μΔw // calculate update term
  w ← w + Δw // weight vector update
  n ← n + B
  if n > N then
    shuffle data
    n ← 1
  end if
until convergence
return w
  
```

Figure 1: High-Curvature Axis (Oscillation Damping) on a steep valley error surface (ill-conditioned terrain)

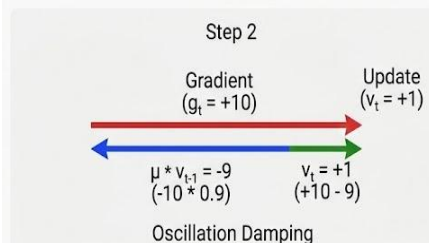
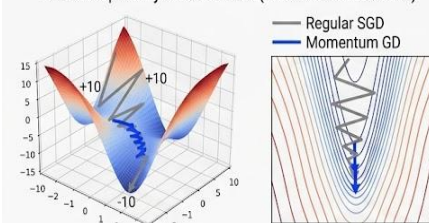
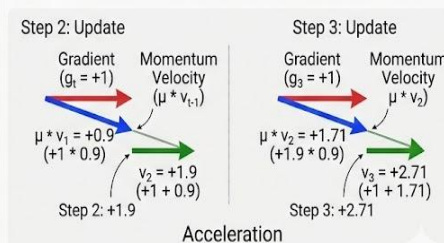
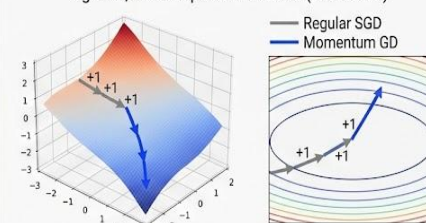


Figure 2: Low-Curvature Axis (Acceleration) a gentle, wide slope error surface (flat terrain)



03 | Advanced Convergence Techniques

Adaptive Optimizers: RMSProp

- 동일한 스칼라 학습률의 한계(각 파라미터 관점)
 - ✓ 모멘텀이 '진행 방향'의 관성을 조절해 준다면, 파라미터별로 직면한 지형의 가파름이 제각각임
 - ✓ η 라는 하나의 스칼라 값 대신 파라미터마다 보폭(Step Size) 자체를 다르게 설정해야 함

- Solution:

- ✓ r_i = 과거 기울기 '제곱'들의 누적 지수 이동 평균

$$w_i = w_i - \frac{\eta}{\sqrt{r_i} + \delta} \frac{\partial E}{\partial w_i}$$

- ✓ 업데이트 식의 분모에 $\sqrt{r_i}$ 를 배치
- ✓ 가파른 축
 - 기울기가 커서 r_i 증가 → 분모가 커지므로 해당 파라미터 전용 보폭 자동 축소 → 진동 없이 안정적 탐색
- ✓ 완만한 축
 - 기울기가 작아 r_i 감소 → 분모가 작아지므로 보폭 자동 확대 → 모든 차원의 탐색 속도를 균일하게 맞춤

03 | Advanced Convergence Techniques

Adam: The De Facto Standard

- Adam (Adaptive Moment Estimation)
 - ✓ 모멘텀의 1차 모멘트(방향성 관성, s) 와
 - ✓ *RMSProp*의 2차 모멘트(보폭 자동 조절, r) 를 완벽하게 결합한 알고리즘
- The Issue: 0 편향(Zero Bias) 문제
 - ✓ 초기 모멘트 변수를 $s = 0, r = 0$ 으로 시작
 - ✓ 첫 스텝에서 실제 기울기가 100이라는 큰 값이 들어와도 수식에 의해 $s_1 = 0.9(0) + 0.1(100) = 10$ 으로 극심하게 쪼그라듐
 - ✓ 이로 인해 학습 극초기에 모델이 움직이지 않는 정체 현상 발생
- Solution: Bias Correction
 - ✓ 보정식 $\rightarrow \hat{s}_i^{(\tau)} = \frac{s_i^{(\tau)}}{1 - \beta_1^\tau}$
 - ✓ 첫 스텝 적용 시 $\rightarrow \frac{10}{1 - 0.9^1} = 100$
 - ✓ 수학적 트릭으로 원래 기울기 크기를 완벽히 복원하여 학습 초기의 멈춤 현상을 해결하고 안정적 수렴을 보장

Algorithm 7.4: Adam optimization

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$
Batch size B
Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
Learning rate parameter η
Decay parameters β_1 and β_2
Stabilization parameter δ

Output: Final weight vector $\mathbf{w}$

```
 $n \leftarrow 1$   
 $s \leftarrow \mathbf{0}$   
 $r \leftarrow \mathbf{0}$   
repeat  
  Choose a mini-batch at random from  $\mathcal{D}$   
   $\mathbf{g} = -\nabla E_{n:n+B-1}(\mathbf{w})$  // evaluate gradient vector  
   $\mathbf{s} \leftarrow \beta_1 \mathbf{s} + (1 - \beta_1) \mathbf{g}$   
   $\mathbf{r} \leftarrow \beta_2 \mathbf{r} + (1 - \beta_2) \mathbf{g} \odot \mathbf{g}$  // element-wise multiply  
   $\hat{\mathbf{s}} \leftarrow \mathbf{s} / (1 - \beta_1^\tau)$  // bias correction  
   $\hat{\mathbf{r}} \leftarrow \mathbf{r} / (1 - \beta_2^\tau)$  // bias correction  
   $\Delta \mathbf{w} \leftarrow -\eta \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$  // element-wise operations  
   $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$  // weight vector update  
   $n \leftarrow n + B$   
  if  $n + B > N$  then  
    shuffle data  
     $n \leftarrow 1$   
  end if  
until convergence  
return  $\mathbf{w}$ 
```

04 | Normalization Techniques

Data Normalization & Internal Covariate Shift

- Data Normalization

- ✓ x_1 (EX. 나이 0~100) 과 x_2 (연봉 0~1억) 등 입력 피처의 스케일이 제각각이면 가중치가 조금만 변해도 에러 표면이 w_2 축 방향으로만 극단적인 베팅이 되는 찌그러진 타원형 지형이 형성

$$\tilde{x}_{ni} = \frac{x_{ni} - \mu_i}{\sigma_i}$$

- ✓ (평균 0, 분산 1) 로 모든 데이터를 표준화하여 등방성(Isotropic, 완벽한 원형) 으로 다듬어 줌

- Internal Covariate Shift

- ✓ 심층 신경망은 역전파 시 연쇄 법칙에 의해 층마다 자코비안 특이값(행렬 크기) 이 누적 곱해지며 기울기 폭발 / 소실이 필연적으로 발생
- ✓ 학습 도중 $l - 1$ 번째 층의 가중치가 업데이트되면 다음 l 번째 층으로 전달되는 데이터 분포 스케일 자체가 매 스텝 널뛰기를 함
- ✓ 다음 층 입장에서는 룰이 계속 바뀌므로 방어를 위해 어쩔 수 없이 학습률 η 를 매우 작게 설정해야 하는 치명적 비효율 발생

04 | Normalization Techniques

Batch Normalization

- 강제 정규화 수식

$$\hat{a}_{n_i} = \frac{a_{n_i} - \mu_i}{\sqrt{\sigma_i^2 + \delta}}$$

- ✓ 각 은닉층 활성화 전(Pre-activation) 단계에서 미니배치 통계량을 기준으로 평균 0, 분산 1을 강제 적용
 - δ = 분산이 우연히 0이 되어 분모가 0이 되는 컴퓨터 에러(ZeroDivisionError)를 막기 위해 더해주는 상수
- ✓ 분포 변동(공변량 이동)을 물리적으로 차단하여 수십 배 큰 학습률 사용 가능

- 비선형성(Non-linearity) 상실

- ✓ 시그모이드(Sigmoid) 등의 함수를 통과할 때 정규화를 하면 데이터 95%가 선형 구간(-2 ~ +2)에 갇혀버림
- ✓ 이는 신경망의 지능인 비선형 표현력을 완전히 파괴

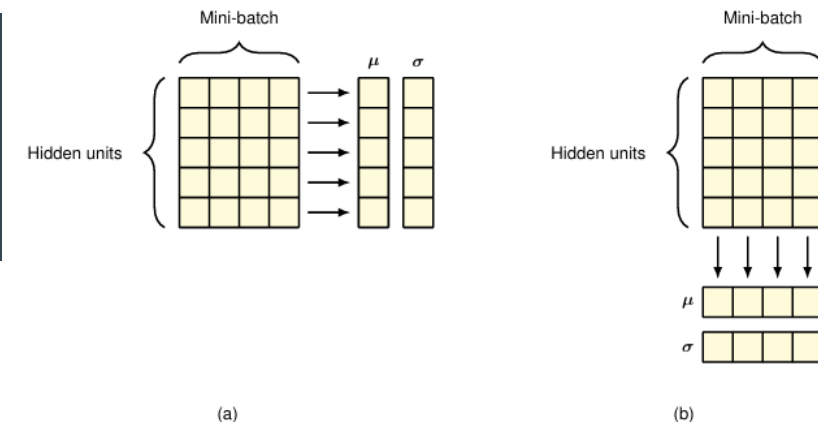
- Solution: 복원 파라미터 도입

$$\tilde{a}_{n_i} = \gamma_i \hat{a}_{n_i} + \beta_i$$

- ✓ 정규화 데이터 진행한 데이터에 모델 스스로 에러를 줄이는 방향으로 γ (스케일 확대/축소), β (좌우 이동) 파라미터를 학습
- ✓ 값이 억눌려 파괴된 비선형 곡선 구간으로 유연성(표현력)을 완벽히 복원

04 | Normalization Techniques

Layer Normalization



- 배치 정규화(BN)의 태생적 한계
 - ✓ 하드웨어 메모리 제약으로 배치 크기가 2~4개 수준으로 매우 작아지면 미니배치의 통계량이 모집단을 대변하지 못해 정규화가 완전히 붕괴
 - ✓ RNN, 트랜스포머(Transformer) 등 문장 길이가 제각각인 시퀀스 모델 에는 배치 축을 묶어 평균을 낼 수 없어 적용 불가능
- Solution: Layer Normalization
 - ✓ 미니배치 축(N) 의존성을 전면 폐기
 - ✓ 단일 데이터 인스턴스 1개가 전체 은닉 노드(Feature) 축(M) 을 기준으로 평균과 분산을 산출

$$\mu_n = \frac{1}{M} \sum a_{ni}$$

- ✓ 배치 크기가 1이든 100이든 완벽히 독립적으로 작동하며 현대 LLM 아키텍처의 필수 표준 기법

Q&A

감사합니다.